
SWEN 90007 24S2 Assignment Part 3 Report

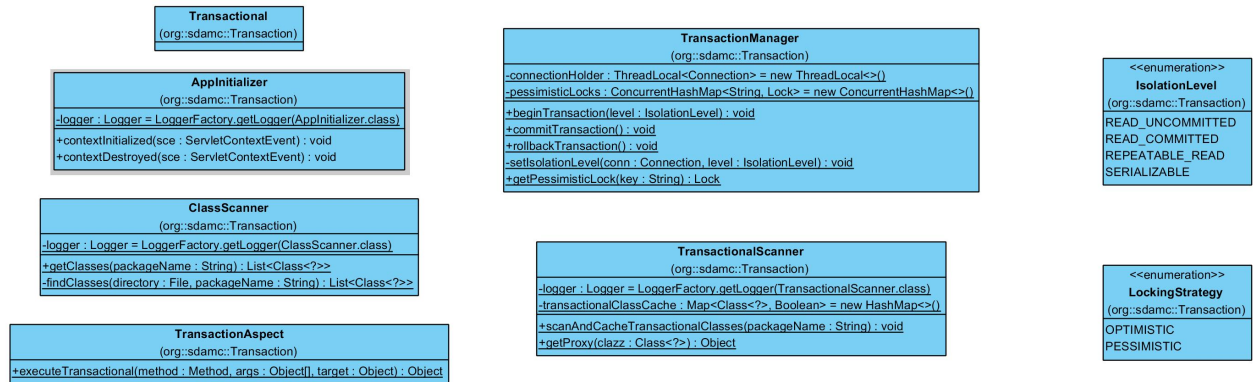
Rong Wang, Pengyuan Yu, Xiang Wang, Yongchun Li

Chapter One Architecture	2
1.1 Additional Class Diagram	2
1.1.1 Transaction part(Loose)	2
1.1.2 Update in RSVP & Funding	2
Chapter Two Concurrency Solution	4
2.1 Event Concurrent amend	4
2.1.1 Solution decision	4
2.1.2 Concurrency Mechanism and Implementation Details:	4
2.1.3 Testing Strategy:	5
2.1.4 considered alternative - Annotated transaction	5
2.2 Simultaneously RSVP	7
2.2.1 Sequence diagram	7
2.2.2 Concurrency Mechanism and Implementation	8
2.2.3 Testing Strategy	8
2.2.4 Other	8
2.3 Funding submit	9
2.3.1 Concurrency Mechanism and Implementation Details:	9
2.3.2 Testing Strategy:	9
2.3.3 Alternatives Considered:	10
2.4 Funding review	10
2.4.1 Concurrency Mechanism and Implementation Details:	10
2.5 Funding modify	11
2.5.1 Concurrency Mechanism and Implementation Details:	11
2.5.2 Testing Strategy:	12
2.5.3 Alternatives Considered:	12

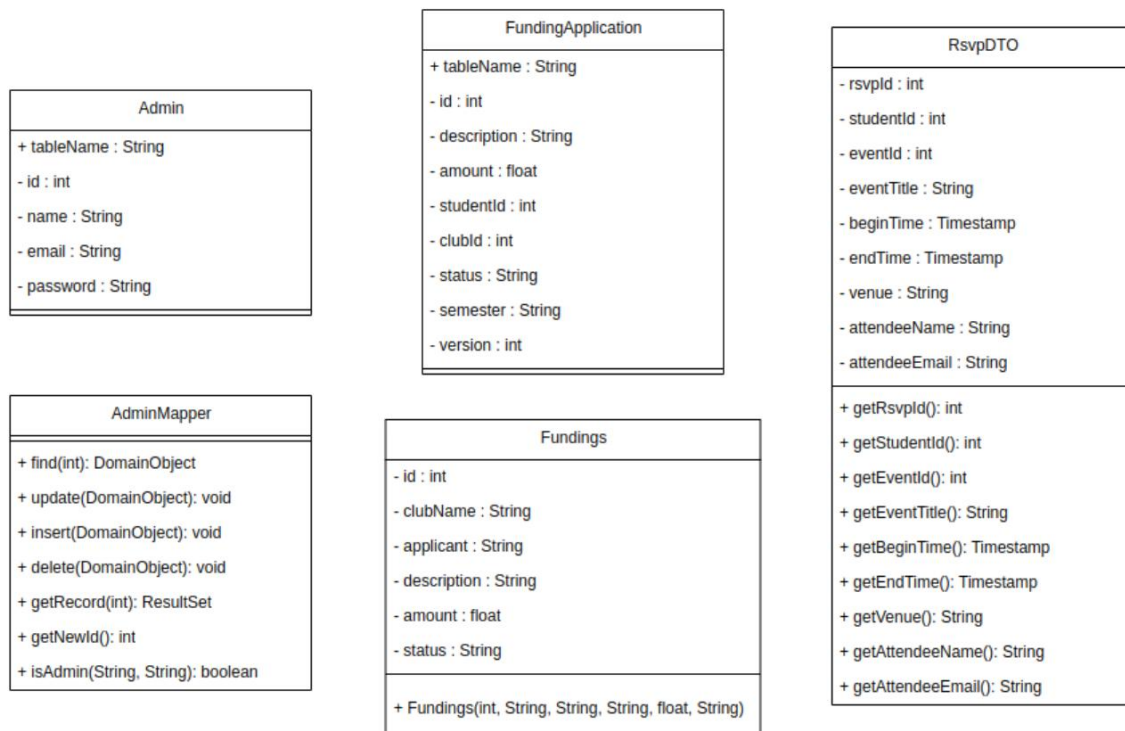
Chapter One Architecture

1.1 Additional Class Diagram

1.1.1 Transaction part(Loose)

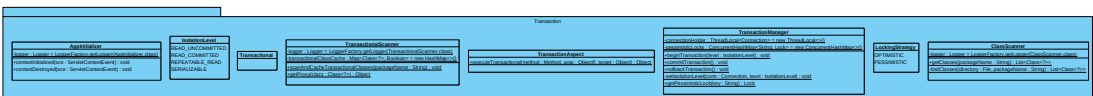
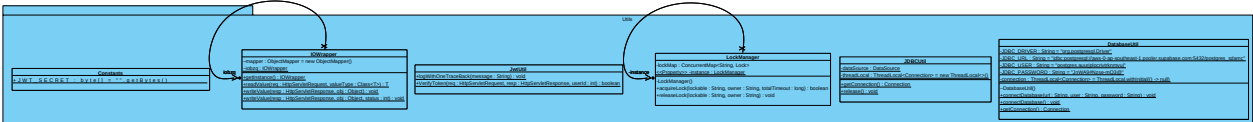
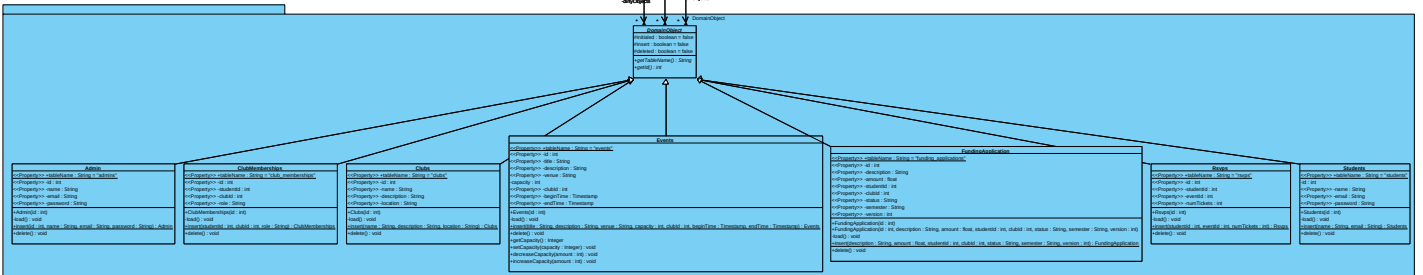
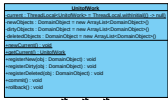
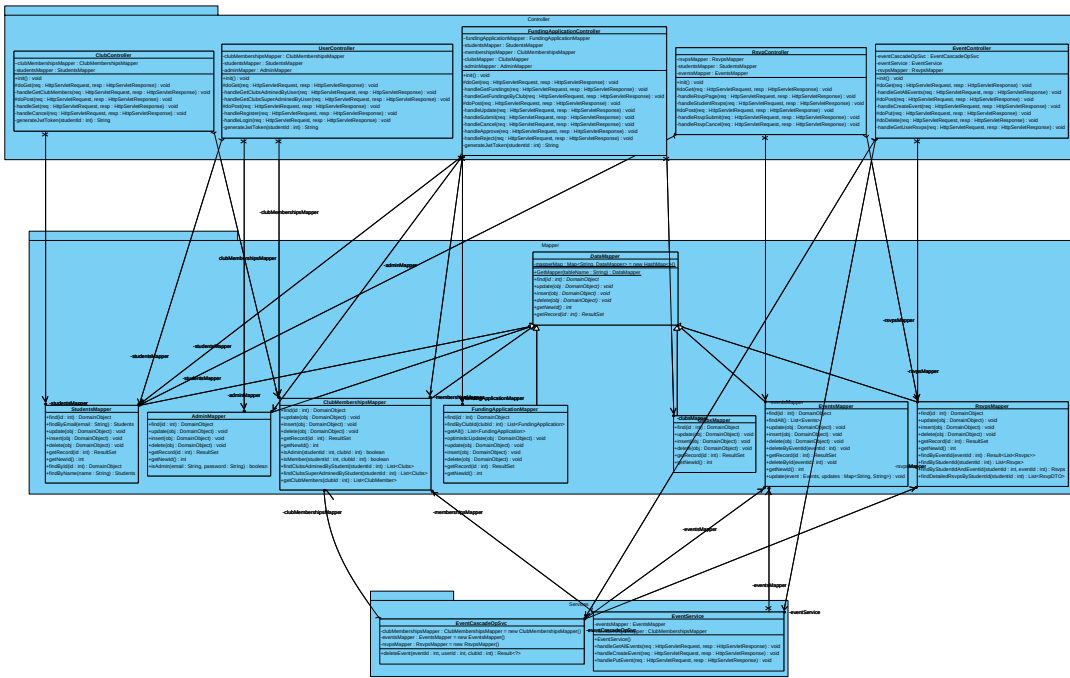


1.1.2 Update in RSVP & Funding



1.1.3 General arch

See next page



Chapter Two Concurrency Solution

2.1 Event Concurrent amend

In a concurrent event modification scenario, club administrators may simultaneously attempt to modify various fields of the same event. Due to factors such as network latency and processing order, the outcome of these concurrent updates is unpredictable and inherently non-deterministic. Without predefined priority, the final persisted change is essentially determined by external factors, such as which request reaches the server first or who has better timing in terms of request processing. To manage this concurrency, we define a time window within which all modification requests are considered equal, and they compete for the priority to persist their changes through race conditions. Outside this time window, any update is considered outdated, as it is based on stale information, and should be retried to reflect the most current state of the event.

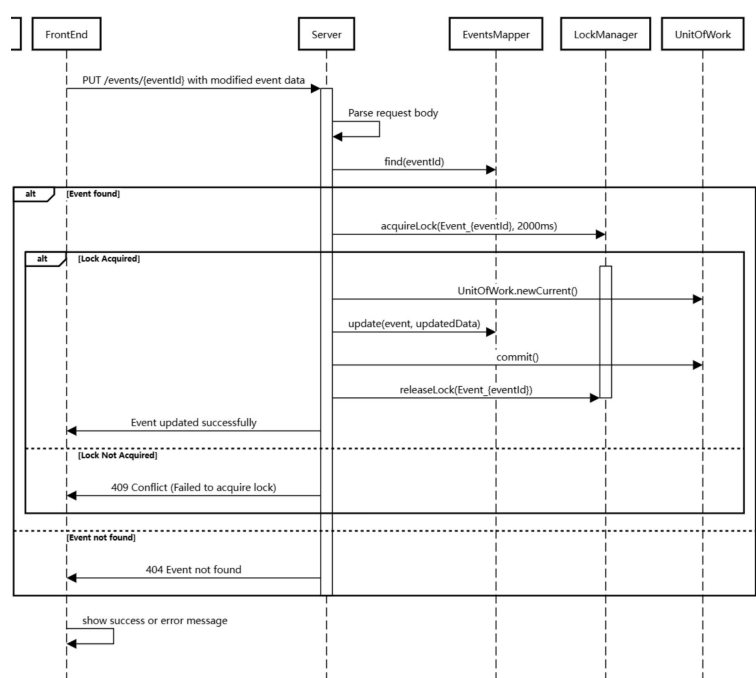
2.1.1 Solution decision

Chosen Solution: ReentranceLock + Exponential Backoff Lock acquire

Classical usage:

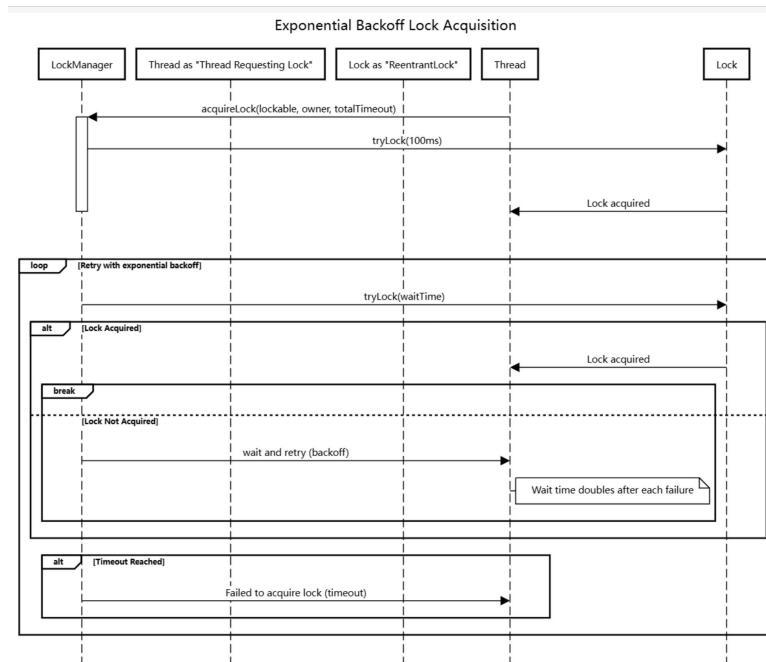
```
boolean lockAcquired = LockManager.getInstance().acquireLock("Event_" + eventId,
"handlePutEvent", 2000);
```

```
if (!lockAcquired) {
    resp.sendError(HttpServletResponse.SC_CONFLICT, " Try again later.");
    return;}
Service logic going on ...
```



2.1.2 Concurrency Mechanism and Implementation Details:

At a macro level, this still falls under the category of pessimistic offline locking—we assume that the updates here are intense and irregular to the point where crafting a custom rollback strategy is extremely difficult. Therefore, we opt to isolate concurrent access from the outset.



Moreover, the service and frontend implementation has been adjusted compared to the previous commit to accommodate new concurrency scenarios—only the fields that have been modified are updated (achieved through dynamic SQL). This prevents overwriting the database with irrelevant old data—although it may seem insignificant under method-level locking, it sets a good foundation for the attempts in 2.1.2.

2.1.3 Testing Strategy:

By using a script to initiate multiple concurrent Curl API requests, with varying fields, the logs show that locks for different event IDs are acquired and released almost simultaneously, while locks for the same event ID are acquired and released in a linear sequence. The wait time for acquiring locks on the same event ID increases exponentially, approximately in multiples of 100 milliseconds, which aligns with the design principles. When the number of concurrent requests increases without adjusting the timeout parameter, timeout errors occur at around 5,800 concurrent requests on a 16-core, 32-thread 5GHz CPU. However, when tested in the Render environment, it only supports around 20 concurrent requests, likely due to the 0.1 core virtual CPU limit.

This test ensures the correctness of reentrance implemented, thus later tests with this lock will focus more on their contextualized content.

2.1.4 considered alternative - Annotated transaction

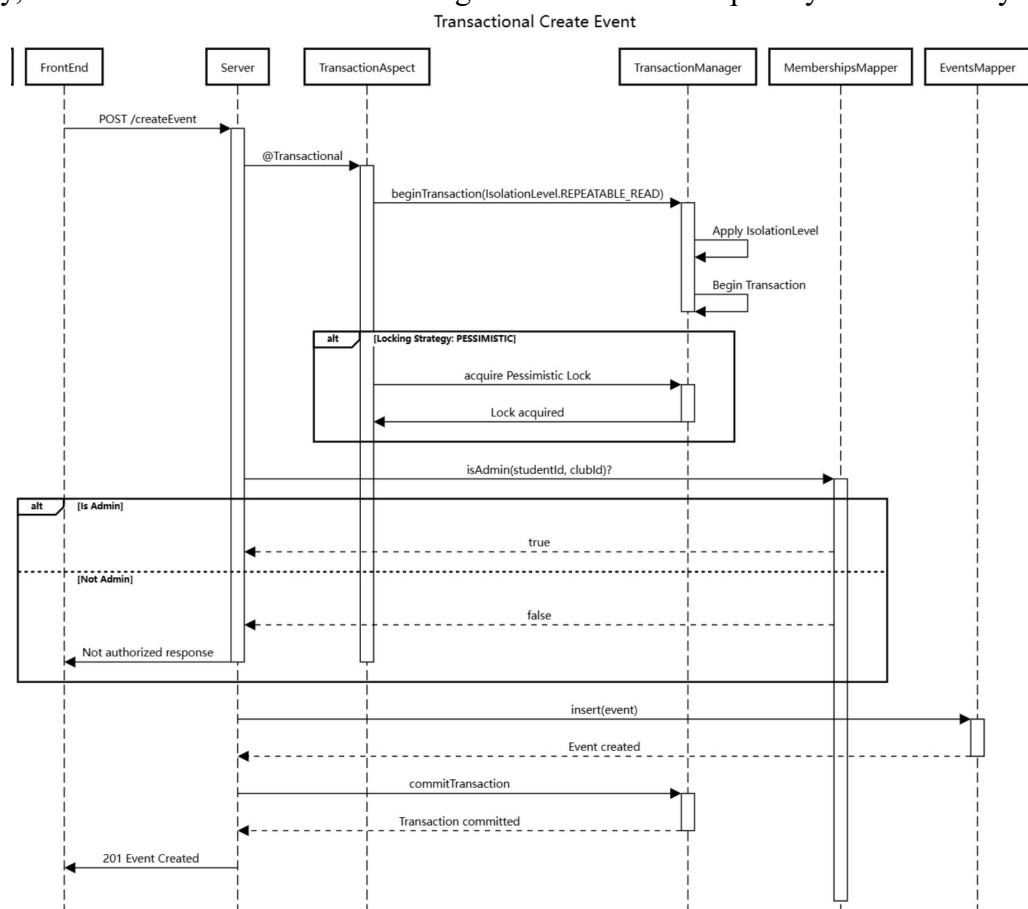
In this implementation, a custom annotation `@Transactional` is registered within the initializer by utilizing a package scanner to identify its usage throughout the application. Upon the annotation'

s activation, the JDBC connection will be temporarily configured to disable auto-commit, ensuring that database operations within the scope of the transaction are not prematurely committed.

The management of the Unit of Work (UoW) is seamlessly abstracted away from the service implementations. Instead of being explicitly handled in the service layer, the transaction's lifecycle, including opening, committing, or rolling back, is orchestrated transparently by a proxy. This proxy is pre-generated during the service's initialization and is automatically substituted for the original service reference at the call site.

A critical aspect of the transaction handling is the proxy's responsibility for locking mechanisms and setting the appropriate transaction isolation levels, aligning with the specific requirements of the annotated method. The process is monitored through an aspect-oriented approach (AOP), where the execution of methods intercepted by the `@Transactional` annotation is evaluated for exceptions or anomalies during runtime. In the event of an exception, the transaction is rolled back to ensure data integrity. Conversely, if the method completes successfully, the transaction is committed, thereby finalizing the operations as part of a consistent unit.

By leveraging AOP and dynamic proxies, this approach encapsulates transaction management within a declarative style, significantly reducing boilerplate code in the service layer while ensuring robust control over transaction boundaries. The ability to manage locks, isolation levels, and rollback scenarios without burdening the application logic allows developers to focus on business functionality, assured that the transaction management is handled implicitly and efficiently.



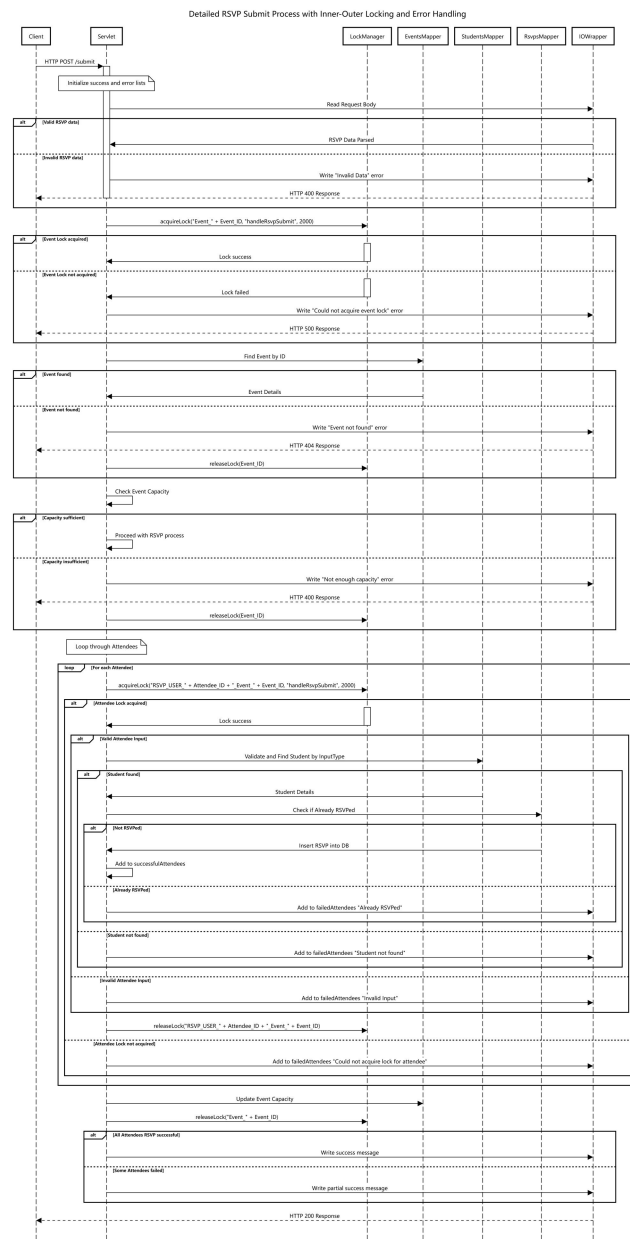
handleCreateEvent with `@Transactional(isolationLevel = IsolationLevel.REPEATABLE_READ, lockingStrategy = LockingStrategy.PESSIMISTIC)`

Rejected reason:

Although we aimed to mimic Spring's `@Transactional` for integrated transaction management, we ultimately found it incompatible with the existing `UnitOfWork` and `Servlet` paradigm. The former uses multiple layers of proxies to seamlessly enforce rollback without explicit awareness, whereas in the basic `Servlet` context, custom annotations cannot intervene in the business logic. A typical issue is that, despite the transaction not being committed, a success message would still be returned. This annotation remains working on Create Events since inserting things are always harmless.

2.2 Simultaneously RSVP

2.2.1 Sequence diagram



2.2.2 Concurrency Mechanism and Implementation

Chosen Solution: Pessimistic Offline Lock

In RSVP function, we employ a pessimistic locking strategy to manage concurrent access to shared resources. Pessimistic locking ensures that when a user attempts to RSVP for an event, the event's capacity is accurately reflected and not overbooked due to simultaneous requests. When multiple users simultaneously attempt to RSVP for the same event in high-concurrency situations, some requests may experience a brief wait due to the event-level pessimistic lock, but the system minimizes waiting times through quick lock releases and an exponential backoff strategy. We also implemented a multi-level locking strategy. This approach utilizes both event-level and user-level locks, managed by a custom LockManager class.

Alternatives Considered:

Optimistic offline lock: Popular events may experience high levels of simultaneous RSVP attempts, increasing the likelihood of conflicts. In such scenarios, optimistic locking could lead to frequent transaction rollbacks and retries, which would be time-consuming and frustrating for users who would need to repeatedly attempt their RSVP. In contrast, our implementation of pessimistic locking, featuring short timeouts and an exponential backoff strategy, is specifically designed to minimize performance impacts.

2.2.3 Testing Strategy

Concurrency Tests: Simulate multiple users attempting to RSVP for the same event simultaneously to ensure:

- ◆ The event's capacity is correctly managed and not overbooked.
- ◆ Only valid RSVPs are recorded, respecting the event's capacity limit.

Timeout and Error Handling Tests: Ensure the system correctly handles scenarios where lock acquisition fails:

- ◆ Verify that RSVP requests that cannot acquire a lock within the timeout period are gracefully rejected.
- ◆ Test the system's ability to provide meaningful error messages to users in case of lock timeout.

2.2.4 Other

The controller implementation has also been updated to keep pace with improvements. We have introduced RSVP simplification, where users only need to specify the required types for reservations, such as ID, name, and email, and provide the specified item.

The system now supports batch RSVPs, with detailed notification for both successful and failed entries. Additionally, the forms for successful submissions are automatically cleared, allowing users to resubmit easily and clearly.

2.3 Funding submit

Administrators can submit one funding application per semester per club. However, if two administrators try to submit applications at the same time for the same club and semester, the system could create duplicate records, violating the rule that only one application should exist per club-semester.

Chosen Solution:

Unique Constraints + Pre-check + Optimistic Handling

2.3.1 Concurrency Mechanism and Implementation Details:

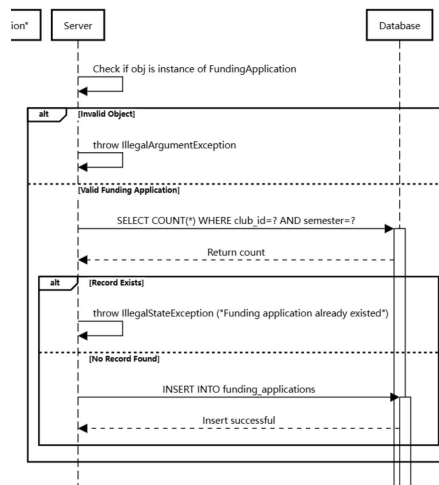
- **Unique Constraint in Database:**

A unique constraint ensures that no duplicate applications are submitted for the same club and semester.

```
ALTER TABLE funding_applications
ADD CONSTRAINT unique_club_semester UNIQUE (club_id, semester);
```

- **Pre-check Query in Application Logic:**

Before attempting an insert, the system performs a pre-check to see if an application for the same club and semester already exists.



```
SELECT COUNT(*) FROM funding_applications WHERE club_id = ? AND semester = ?;
```

- **Handling Duplicate Submissions:**

If two administrators submit applications simultaneously, the unique constraint will raise an SQL exception for the second insert.

2.3.2 Testing Strategy:

1. Concurrency Tests: Simulate multiple administrators submitting applications simultaneously to ensure only one application is recorded.
2. Rollback Verification: Ensure the second submission fails gracefully with a clear error message for the user.
3. Performance Tests: Verify that the unique constraint does not cause performance degradation under normal use.

2.3.3 Alternatives Considered:

Pessimistic Locking: Rejected because it would block other administrators unnecessarily, leading to poor scalability.

2.4 Funding review

Faculty administrators review, approve, or reject funding applications. If two administrators review the same application simultaneously, it could lead to conflicting status changes.

Chosen Solution: Pessimistic Locking

2.4.1 Concurrency Mechanism and Implementation Details:

- **Lock the Record with SELECT FOR UPDATE:**

To prevent simultaneous modifications, the system locks the application record while a review is in progress.

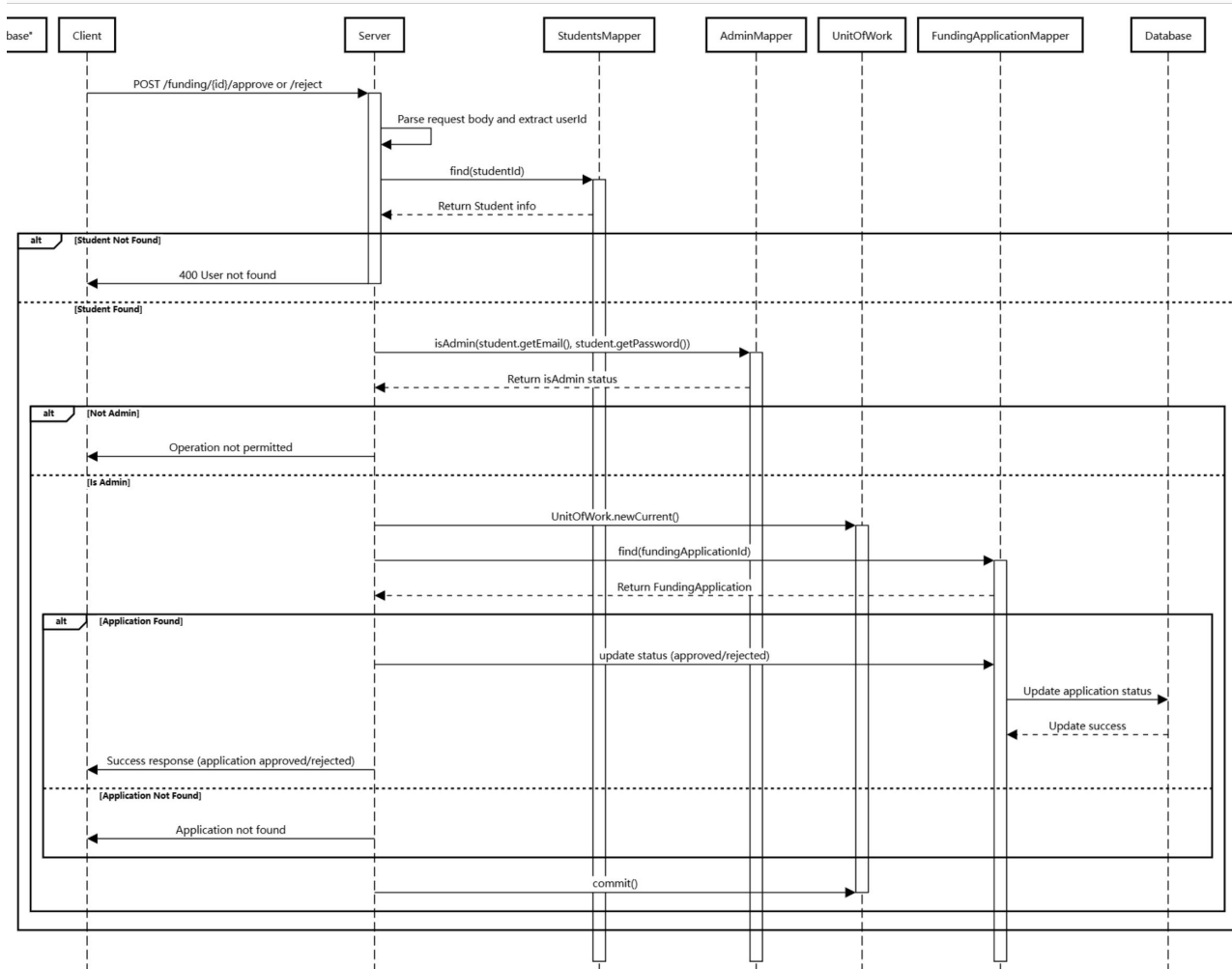
```
BEGIN;  
SELECT * FROM funding_applications WHERE id = ? FOR UPDATE;  
UPDATE funding_applications SET status = ? WHERE id = ?;  
COMMIT;
```

- **Transaction Management:**

Operations within the review process are wrapped in a transaction to ensure atomicity.

- **Timeout Mechanism:**

To avoid deadlocks, the lock will timeout if held for too long.



● Testing Strategy:

1. Multi-threaded Testing: Simulate two administrators attempting to review the same application to ensure only one can proceed.
2. Deadlock Testing: Introduce artificial delays to verify the system handles timeouts gracefully.
3. Correct Status Validation: Verify that only the final status is recorded after a review.

● Alternatives Considered:

Optimistic Locking: Rejected because frequent retries degrade user experience.

2.5 Funding modify

Administrators may modify funding applications. If two administrators edit the same application concurrently, the system could lose one administrator's changes without warning.

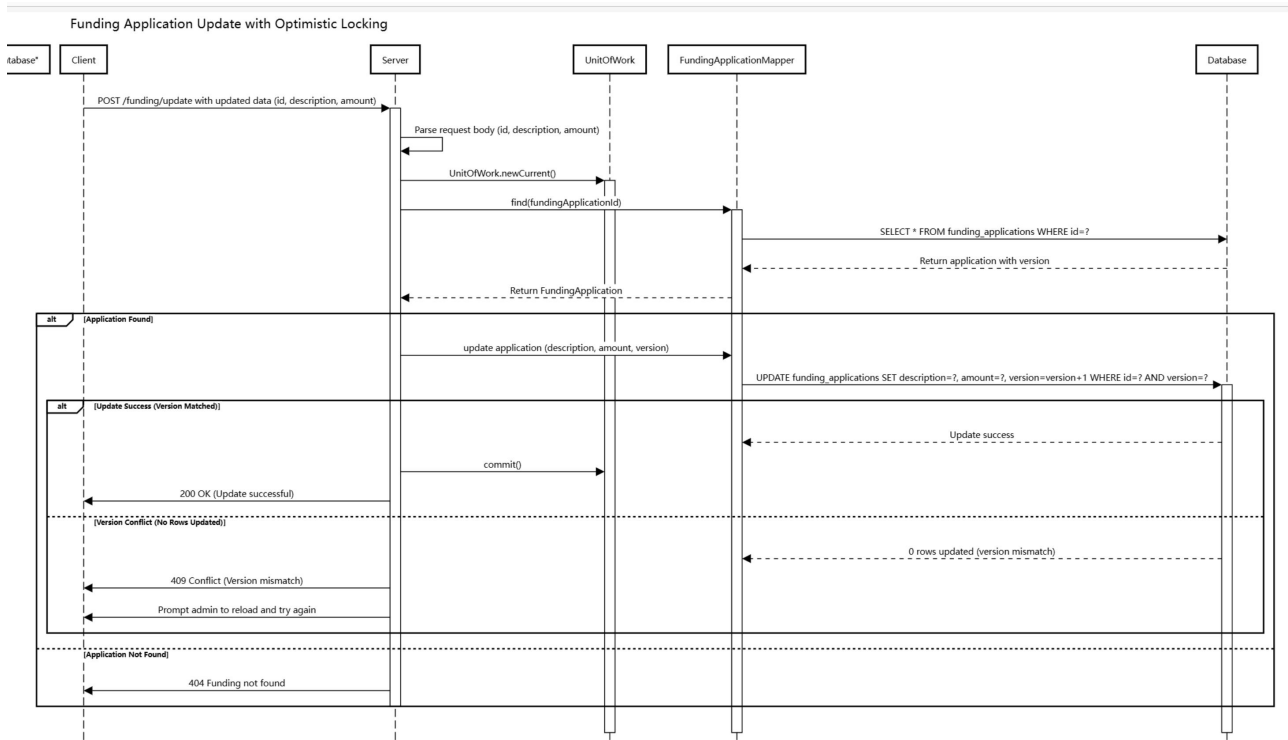
Chosen Solution: Optimistic Locking

2.5.1 Concurrency Mechanism and Implementation Details:

● Add a version Column to Track Changes:

```
ALTER TABLE funding_applications ADD COLUMN version INT NOT NULL DEFAULT 1;
```

● Update Query with Version Check:



```
UPDATE funding_applications SET description = ?, amount = ?, version = version + 1 WHERE id = ? AND version = ?;
```

● Error Handling for Version Conflicts:

Before we update the row in database, we need to record the current version number. When we are set to manipulate the database, we are supposed to lock and check whether the versions are the same. If no rows are affected at the end, the system prompts the user to reload the data and try again.

2.5.2 Testing Strategy:

1. Concurrent Update Simulation: Create tests where multiple administrators attempt to modify the same application simultaneously.
2. Version Increment Validation: Verify that the version number increments correctly after each successful update.
3. Conflict Handling Tests: Ensure users receive appropriate conflict messages when a version mismatch occurs.

2.5.3 Alternatives Considered:

Pessimistic Locking: Rejected because it would block reads, reducing performance and scalability.